从AI Coding到Harness Engineering的端到端工程开发实践

原创 腾讯程序员 腾讯技术工程 Jul 3, 2026, 2:36 AM 广东 191 listened



作者：zimingxing、kinglongli 、yifhao

应用宝活动平台系统支撑应用宝内包括 app、pc、手助等产品的所有日常/节假日活动，在今年上半年，我们针对整套系统进行了一次完整的重构。正值这个时间点，Harness Engineering 的概念被提出，我们团队也在重构的过程中，针对新的活动平台系统引入 Harness Engineering 的工程实践，初步搭建了一套 AI 端到端的开发流程。 这篇文章主要记录了团队在践行 Harness 工程化过程中遇到的一些问题以及实践经验，抛砖引玉，欢迎其他团队同学一起探讨。

一、为什么要实践 Harness Engineering

在最初的开发阶段，我们主要采用对话式 AI coding 的方式进行开发，通过 CodeBuddy + Plan Mode、Rules + Prompt、AI 辅助编码、人工 Review 这样的协作方式让项目跑起来。

这种开发方式在实现一些一次性任务、或者小需求的时候会非常快，但随着项目复杂度的提升，逐渐暴露出了几个明显问题：

1.1 单窗口上下文快速膨胀

为了让 AI 能清晰的知道我们的业务背景、代码规范、项目结构等等，往往我们每次都需要在给 AI 的 prompt 中输入这些信息。

为了避免每次重复输入这些信息，我们会将团队的一些背景知识、代码规范、生成规则等沉淀为文档或 rules，在每次对话中塞到上下文带给 AI。



随着项目迭代，知识背景、规则会越来越多，每次启动上下文窗口，还没输入需求就已经塞了很大一部分内容。

另外，如果在单一窗口下让 AI 实现需求，往往多轮交互下来，上下文会快速膨胀，出现有损压缩导致遗忘、不按规则生成等等情况。

1.2 缺乏完整的业务知识

每次需求开发都依赖人在 Prompt 中讲清楚需求背景、涉及业务的知识等等信息，而一个需求往往可能涉及多个业务、多个服务，这些服务之间如何合作、如何实现串联，都需要人来捋清楚后喂给 AI，一个完整的上下文构造非常耗费精力。

1.3 缺乏工程上的自动化闭环

对话式 AI coding 的方式仅仅只能在 coding 环节由 AI 实现，但一个需求的完整交付，还包括了需求分析拆解、测试环境部署、接口验证、测试等等环节，这些环节依旧需要依赖人来执行和验证。如果希望在这些环节也都能让 AI 参与，真正做到 AI 提效，就需要从工程上实现全流程的 AI 能力集成，实现从需求到上线的闭环。

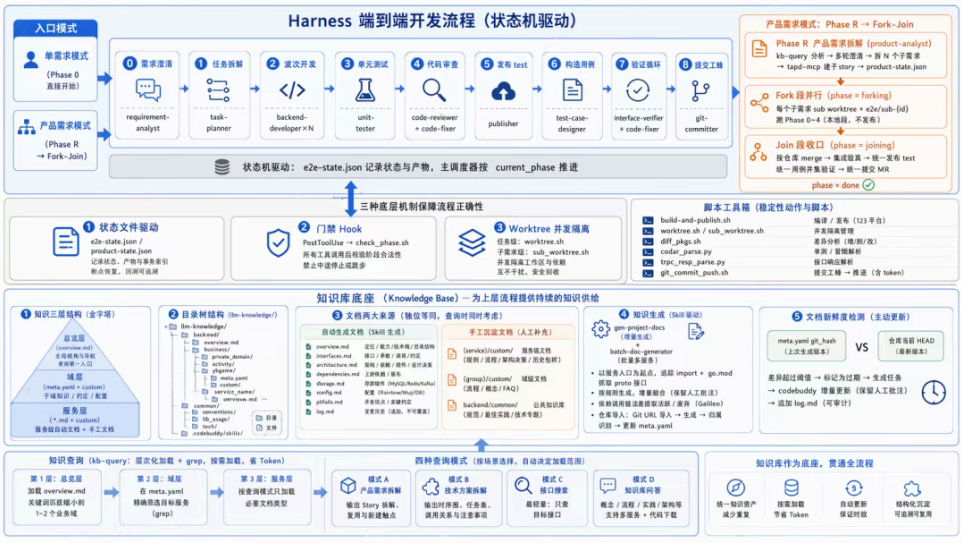
1.4 单窗口无法并行

在一个窗口内对话，一次只能执行一个任务，当需求涉及多个独立的开发任务（例如开发三个独立接口），要么只能一个一个让 AI 开发，要么就只能同时开三个窗口，分别跟 AI 对话让其执行。同时，人需要在其中不断切换窗口进行对话，以推动流程往下推进。

总的来说，当任务越是完整、越是规模化、越是可抽象成稳定流程，其仅仅采用对话式 AI coding 来完成的弊端就越明显，采用 Harness 工程实践的价值也就越高，这也正是我们希望从对话式 AI coding 走向 Harness 工程化的原因。同时，我们也想借此探索一下，在真实业务研发场景中，当下大火的 Harness 工程究竟该如何落地实践。

二、整体架构：知识库工程 ✖ 端到端开发工程

应用宝活动平台的 Harness 工程，主要包含了两部分工程能力：知识库工程和基于知识库之上构建的端到端开发工程。



知识库工程负责将散落在代码、文档、可观测系统中的工程知识，按照一定结构进行沉淀和更新，并提供给上层端到端开发工程消费。

整个端到端开发工程以“状态文件”为驱动，实现了将一个需求由 AI 按照 需求拆解->需求澄清->任务拆解->并行开发->单测校验->代码审查->测试环境部署->用例请求构造->接口验证->提交代码 全流程执行，人在其中只负责核心关键阶段的决策和信息补全。

当前，我们的整套体系包含 800+ 结构化文档，知识范围涵盖后台 90+ 个微服务，端到端流程沉淀了 12 个专家 Agent、30+ 业务相关 Skill 以及 10+ 个固定流程脚本。

可以看到，知识库工程是整个 Harness 体系的底座——上层端到端开发流程的每一个环节，从需求拆解到接口验证，都依赖它提供准确、结构化的业务上下文。因此，本文会先从知识库工程开始介绍，再展开到端到端开发工程。

三、Knowledge Engineering——让 AI 真正懂业务

在复杂业务系统中，AI 最大的问题并不是“不会写代码”，而是缺乏真实、准确、持续更新的业务知识作为背景上下文。

前面提到，对话式 AI Coding 的一个核心痛点是缺乏完整的业务知识——每次开发需求，都需要人先把需求涉及的多个服务、业务背景、接口调用关系等捋清楚，再逐条喂给 AI。更关键的是，这份上下文是一次性的，这次讲清楚了，下次换个需求、换个窗口，又得从头再来。如果没有一套系统化的知识管理机制，AI 在每次执行任务时都相当于"从零开始理解业务"。

因此，知识库是整个 Harness 工程体系的基石——它决定了 AI 在需求拆解、技术方案、代码生成、接口验证等每一个环节中，能否拿到正确、充分的上下文。知识库建设得好不好，直接影响到整个端到端开发流程的整体效果。但是，知识库体系的建设，目前也是业界的一个比较大的难点，在我们团队实践的过程中，其难点主要有以下几个：

- **冷启涉及的知识数据规模大**：应用宝游戏研发后台业务覆盖了游戏宝券/礼包、活动平台、私域运营、微下载等多个业务域，维护着数百个微服务，每个服务涉及多则十几二十个接口，仅接口文档这块，其梳理、编写涉及的工作量就非常大，更不用说还有业务逻辑、领域知识等内容。
- **大量知识背景下如何精确提取知识**：在大量业务知识的情况下，针对每一次问答，如何用好知识，准确查询出需要的内容，也是一个需要解决的问题。
- **知识更新迭代速度快，已有知识容易过期**：随着产品的持续迭代，无论是代码还是业务形态的更新都非常快，在以前的文档中，代码改了但文档没改的情况几乎是常态。一份写于半年前的业务梳理文档，往往已经和现在的逻辑严重脱节。

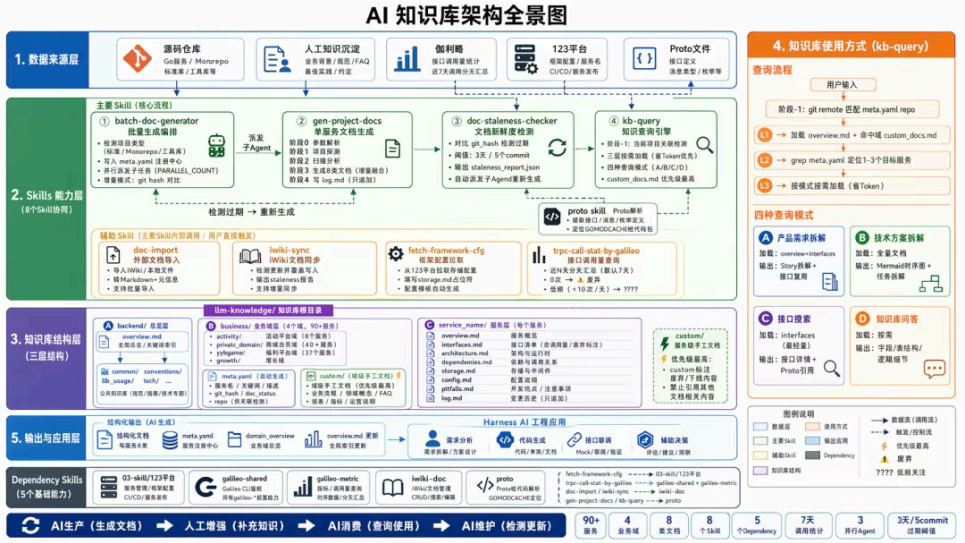
业界当下对于知识库如何组织，也有非常多的讨论，LLM Wiki、Obsidian-Wiki、GBrain 等等或方法论或实践方案也层出不穷。

这里，借鉴 LLM Wiki 和 Obsidian-Wiki 的方法论和实践，我们团队搭建了一套结构化的知识库，由两部分协同构成：

- 自动生成部分：通过 Skill 体系让 AI 自动读代码、自动生成结构化文档、自动维护新鲜度。

- 手工沉淀部分：由人在 `custom.md` 和 `common/` 中沉淀业务背景、架构决策、使用规范等 AI 难以从代码中提取的知识。

两者结合形成"AI 生产 + 人工补充 + AI 消费"的知识库体系，为 Harness 工程提供持续更新的知识信息。



3.1 整体目录结构

```
llm-knowledge/
├── backend/
|   ├── overview.md                # 全局总览
|   ├── business/
|   |   ├── private_domain/        # 商城会员域
|   |   ├── activity/              # 活动平台域
|   |   ├── yybgame/               # 福利平台域
|   |   |   ├── meta.yaml          # 服务索引
|   |   |   ├── custom/            # 域级手工文档
|   |   |   └── service_name/
|   |   |       ├── overview.md    # 等8类自动生成
|   |   |       └── custom/        # 服务级手工文档
|   |   └── ...                    # 更多业务域
|   └── common/                    # 公共知识库
|       ├── conventions/           # 开发规范
|       ├── lib_usage/             # 常用库指南
|       └── tech/                  # 技术专题
└── .codebuddy/skills/
```

整个知识库结构分为三层：

- 总览层 `backend/overview.md`：描述所有业务域的关键词和服务范围，知识库查询的第一入口
- 域层 `{group}/meta.yaml + {group}/custom/`：机器可读的服务索引 + 业务域级别的手工沉淀文档

- 服务层 `{service}/*.md + {service}/custom/`：每个服务 8 类自动生成文档 + 手工补充文档

同时，文档文件分为两类，一类是人工手动沉淀的文档， 另一类则是通过 Skill 自动生成的文档：

手工沉淀文档：

位置	用途
`{service}/custom/` 或 `{service}/custom.md`	服务级手工文档：业务背景、调用示例、架构决策、历史包袱说明等 AI 难以从代码推断的知识
`{group}/custom/` 或 `{group}/custom.md`	业务域级手工文档：跨服务的业务流程说明、领域概念定义、常见问题汇总
`backend/common/`	跨业务域公共知识库：开发规范（`conventions/`）、常用库使用指南（`lib_usage/`）、技术专题（`tech/`）

自动生成文档：

文档	面向读者	内容
`overview.md`	所有人	定位、核心能力、技术栈、目录结构
`interfaces.md`	使用方 / AI	所有接口、参数、业务说明、近7天调用量
`architecture.md`	开发者 / AI	核心数据流、定时任务、Kafka 消费者、关键设计模式
`dependencies.md`	开发者 / AI	上游服务依赖及调用方式
`storage.md`	开发者 / AI	MySQL Redis Kafka 等存储组件
`config.md`	运维 / 开发者	Rainbow Wuji DB 动态配置
`pitfalls.md`	开发者 / AI	开发坑点，含位置、描述、避坑建议
`log.md`	所有人	文档变更历史，每次生成追加，不可覆盖

手工沉淀文档与自动生成文档在知识库中地位等同， `kb-query` 在查询时会同时考虑两类来源。

3.2 知识生成：从代码到结构化文档的自动化流水线

在知识的生成上，我们定义了两个 skill： `gen-project-docs` 和 `batch-doc-generator` ，分别支持单服务的文档生成与批量多服务文档生成。

在生成时，会以服务入口文件为起点，通过 `import` 追踪找到所有接口注册点，再从 `go.mod` 定位 proto 包提取接口定义。同时文档生成会按每类文档的规范模板写作，并进行增量融合——如果服务已有文档，新生成的内容会保留人工补充的业务注释，只更新代码层面的事实性信息（接口增删、参数变更）。

为了明确当前业务接口是一个线上实际在用的接口，还是一个已经废弃或低频使用接口，我们以“**接口调用量**”为指标，在知识文档生成前，查询其伽利略上的调用信息，判断接口是否活

跃，并在文档中进行标注。

另外，知识库会以 `meta.yaml` 作为注册中心， 每个服务的文档状态、 `git hash` 记录等信息都记录在这里。skill 会根据 `git hash` 变更判断是否需要重新生成（增量模式）。

对于全新增的知识，知识库支持仓库导入模式：直接从 Git URL 导入尚未纳入知识库的仓库，将其 clone 到本地、生成文档、分析与现有业务域的归属关系、更新 `meta.yaml`，整个流程自动完成。

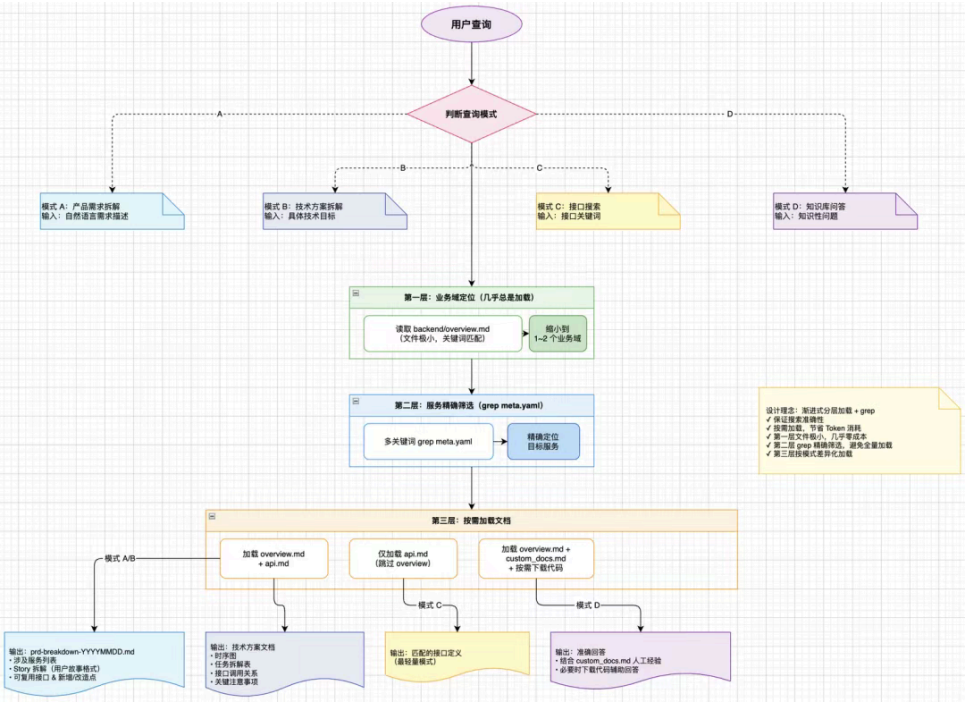
3.3 知识检索：渐进式加载

知识的提取查询也是一个在业界经常被讨论的点，现在似乎有一种共识：**采用层次化加载结合大模型探索的方式替代传统的 RAG 检索**。

之所以这种方式比传统 RAG 更适合知识库的建设，是因为它复刻了人类专家查阅和学习复杂资料的真实认知习惯：

1. **精准定位而非模糊匹配**：标准 RAG 依赖向量算相似度，往往不够精确。层次化加载基于知识的真实逻辑与引用关系进行寻址，能剔除无关信息，避免污染大模型的上下文。
2. **自顶向下的按需吸收**：人类学习总是先抓框架再看细节。通过“全局大纲-核心摘要-底层明细”的知识金字塔，让大模型顺着脉络按需加载，既不会在海量长文本里迷失，又能大幅节省 token。
3. **变“被动查”为“主动探索”**：RAG 往往是一次性被动检索，而探索机制让 Agent 能像专家研究课题一样，根据初步线索在知识网络里顺藤摸瓜、多步跳转，自主拼凑出最精准的上下文。
4. **向量库系统维护成本高**：知识库天天都在更新，维护一套 RAG 向量库的同步成本较高。直接依托结构化的知识本体进行探索，系统架构也更加轻盈。

在我们的知识库工程中，同样是采用**渐进式分层加载+grep 的方式**来实现对结构化知识的检索，既能保证搜索的准确性，同时又能实现按需加载， 节省 Token 消耗。



当下的设计为：

- 第一层几乎总是加载（文件很小），通过关键词匹配缩小到 1-2 个业务域
- 第二层用 `grep` 在 `meta.yaml` 里精确筛选，避免加载整个域的服务文档
- 第三层根据查询模式只加载必要的文档类型——接口搜索连 `overview.md` 都不读。

同时，提供了四种查询模式：

- **模式 A：产品需求拆解。** 用于将 PRD[Q] 分解为可执行的后端 Story。输入是自然语言需求描述，输出是 `output/prd-breakdown-YYYYMMDD.md`，包含：涉及服务列表、Story 拆解（用户故事格式）、每个 Story 可复用的现有接口和需要新增/改造的点。
- **模式 B：技术方案拆解。** 用于针对具体技术目标生成技术方案文档。输出包含时序图、任务拆解表、接口调用关系、以及关键注意事项。
- **模式 C：接口搜索。** 最轻量的模式，适合"找到对应接口就行"的场景。
- **模式 D：知识库问答。** 适用于询问业务概念、业务流程、字段含义、表结构、逻辑细节等知识性问题，按需加载多个服务的文档，结合 `custom_docs.md` 中的人工经验给出准确回答。对于细节性问题，还可以自动下载代码进行回答。

3.4 文档新鲜度检测：过期知识比没有知识更危险

除了生成和查询，知识库工程还有一个问题，便是知识更新迭代速度快，已有知识容易过期。**过期知识比没有知识更危险**，如果 AI 引入了一份过期的知识，会导致由于误导性知识影响整体的流程，而且往往排查起来非常困难。

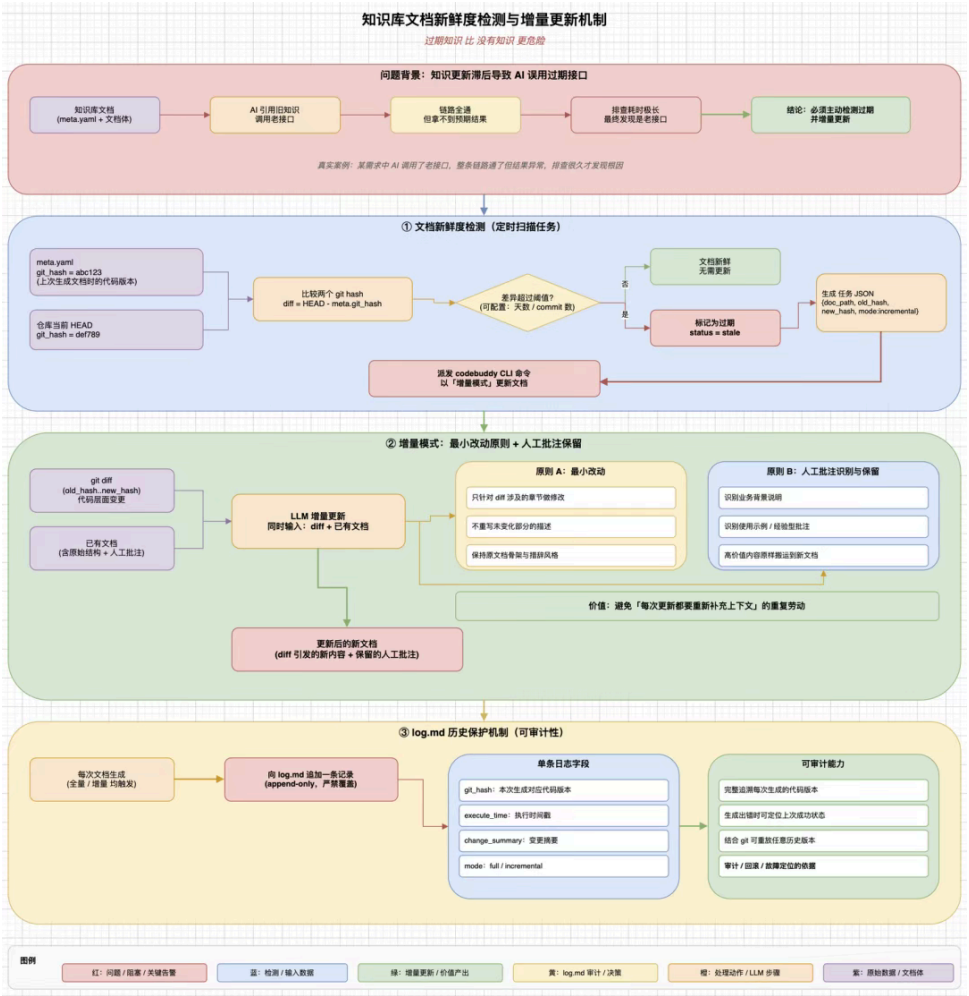
我们实际出现过这样一个例子：在一次开发需求中让 AI 调用一个接口，由于知识库没有更新，AI 调用了一个老的接口，整个链路都通了，但是我们一直拿不到预期的结果，后面花了很久的时间排查，才发现是调用到了老的接口上去。

对此我们引入了文档新鲜度检测任务，来实现针对知识库知识的新鲜度检测。

其工作原理是比较两个 `git hash`：`meta.yaml` 中记录的 `git_hash`（上次生成文档时的代码版本）和仓库当前 `HEAD` 的 `hash`。当差异超过阈值（可配置，如超过一定天数或 commit 数）时，该文档会被标记为过期，生成任务 json，然后派发 codebuddy 命令行以增量模式更新生成文档。

增量模式的关键特性是**最小改动原则+人工批注保留**：增量生成会把 git diff 的内容喂给大模型，同时让模型读取已有文档，根据 diff 内容对已有文档进行修改，尽量减少对既有文档进行大幅度改动。同时会识别其中人工添加的业务背景说明、使用示例等“高价值内容”，在新文档中予以保留。这避免了“每次更新都要重新补充上下文”的重复劳动。

同时，添加了 `log.md` 历史保护机制确保可审计性：每次生成都向 `log.md` 追加一条记录（含 `git_hash`、执行时间、变更摘要），严禁覆盖历史条目。即使某次生成出错，也能通过 `log.md` 结合 `git` 追溯到上次成功状态。



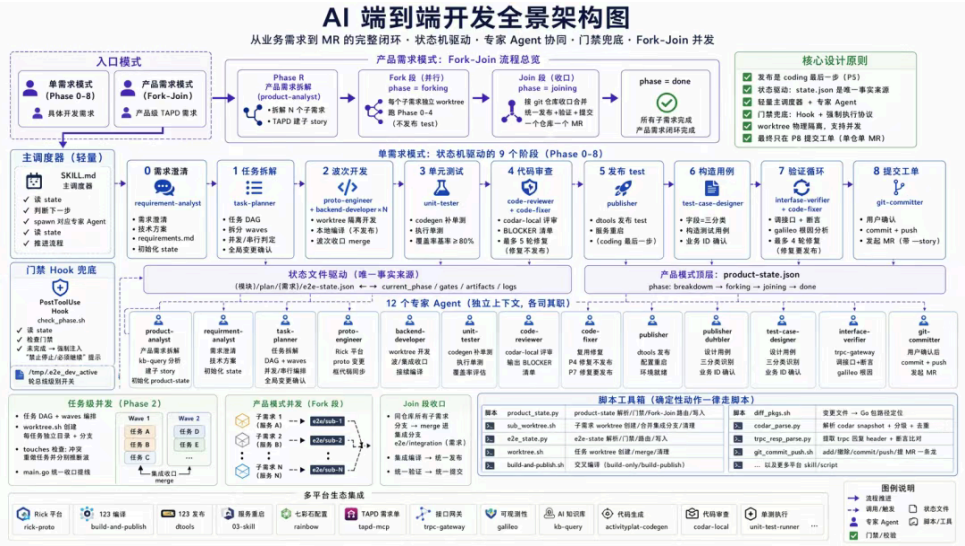
目前，这套知识库工程已经覆盖了应用宝游戏后台包括活动、福利、商城、增长等在内的 4 个业务领域，涵盖 90 多个微服务，800+份结构化文档。

四、端到端开发工程建设——从"写代码"到"交需求"

要突破对话式 AI Coding 的瓶颈，其核心不在于模型是否足够聪明，而在于将原来的开发模式从“一个对话窗口”，升级为一套工程系统，也就是现在 Harness 工程的一个核心思想。在我

们的端到端开发工程中，围绕 Harness Engineering 的设计思想，我们主要做了几个事情：

- **拆分上下文替代单窗口累计**：把一个长的开发流程拆分成多个聚焦的、相对独立的子任务，由单独的子 agent 来执行，每个子 agent 只加载它当前所需要的信息。
- **以状态文件替代容易被压缩的对话历史**：把整个开发流程中，进行到了哪一步，每个步骤的产出等信息持久化下来，让整个执行流程具备可观测、可中断、可恢复、可协同的能力。
- **用确定性编排替代概率性发挥**：把需求拆解→开发→单测→代码评审→部署→接口验证等等流程固化为按序推进、带质量验证的流水线，每一步骤不达标则不往下推进，把开发流程推进从“靠人推动”变成"靠流程保证"。
- **用 DAG + 多 Agent 并行替代单线程串行执行**：自动识别任务间的依赖关系，将任务编排为 DAG 拓扑结构，把同一层的任务分发给并发 agent 在隔离环境中同时开发，再统一收口，实现多任务的并行执行。
- **打通从开发到提交工蜂代码全流程**：通过集成或者封装司内包括 TAPD、123、03、rick、伽利略、七彩石、无极等等各大平台的能力，打通整个需求实现中各个环节和外部依赖。



4.1 状态文件驱动：让流程脱离对话窗口而独立存在

在对话式 AI Coding 中，整个开发过程依赖对话历史来传递上下文。随着开发推进，对话历史会迅速膨胀，出现三个典型问题：

1. 上下文有损压缩导致 AI 遗忘早期信息或偏离既定规则；
2. 流程中断后无法恢复——一旦对话窗口关闭或上下文被截断，之前的所有中间状态就都丢失了；
3. 流程推进缺乏可观测性，人和 AI 都不知道现在执行到哪一步了。

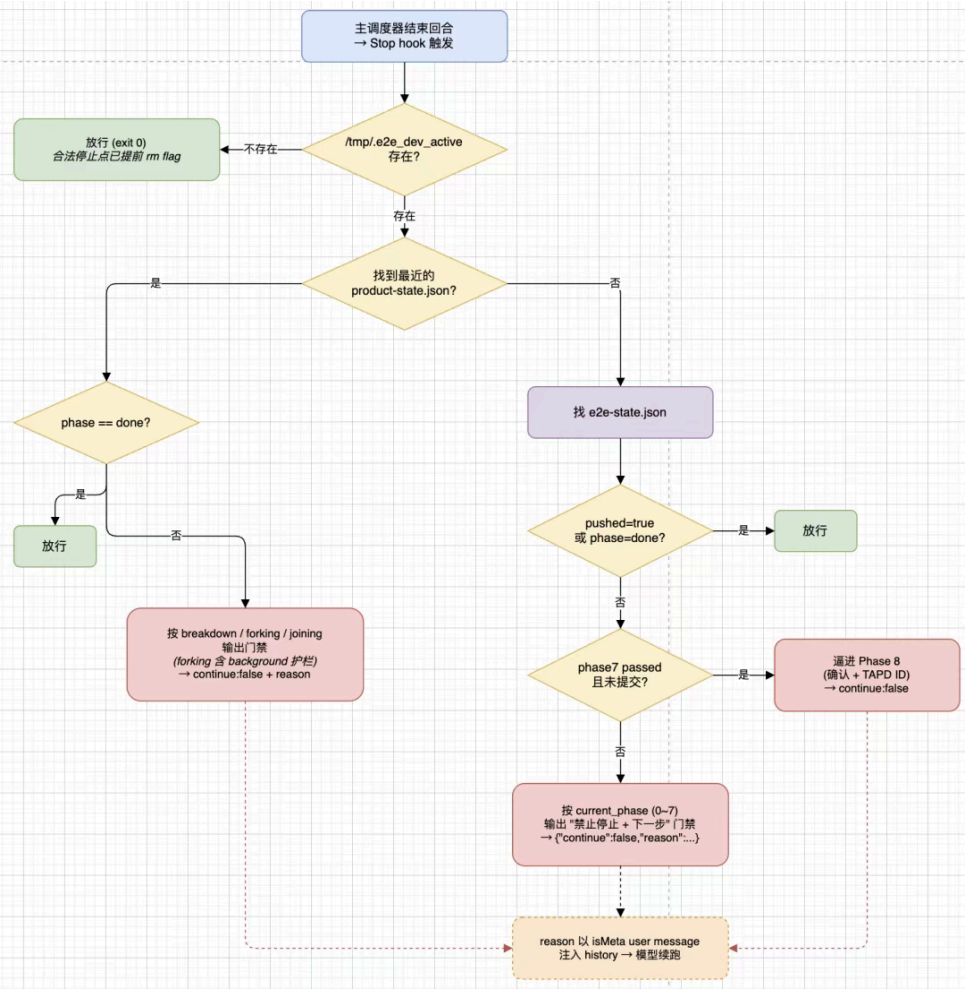
要解决这些问题，一个核心思路就是：**长链路必须状态化**。把开发流程中的每一步进度、产出、依赖关系持久化到文件中，让上下文脱离对话窗口独立存在。

4.1.1 状态文件：流程的唯一真相源

整个端到端开发流程的状态被持久化在两类结构化 JSON 文件中：

- `product-state.json`：用于产品需求拆解后的多 Story 并行开发流程，记录拆解阶段（breakdown）、并行分叉阶段（forking）、合并收口阶段（joining）等状态；
- `e2e-state.json`：用于单 Story 的端到端开发流程，记录当前所处的 Phase（0~7）、每步的产出、接口验证结果等。

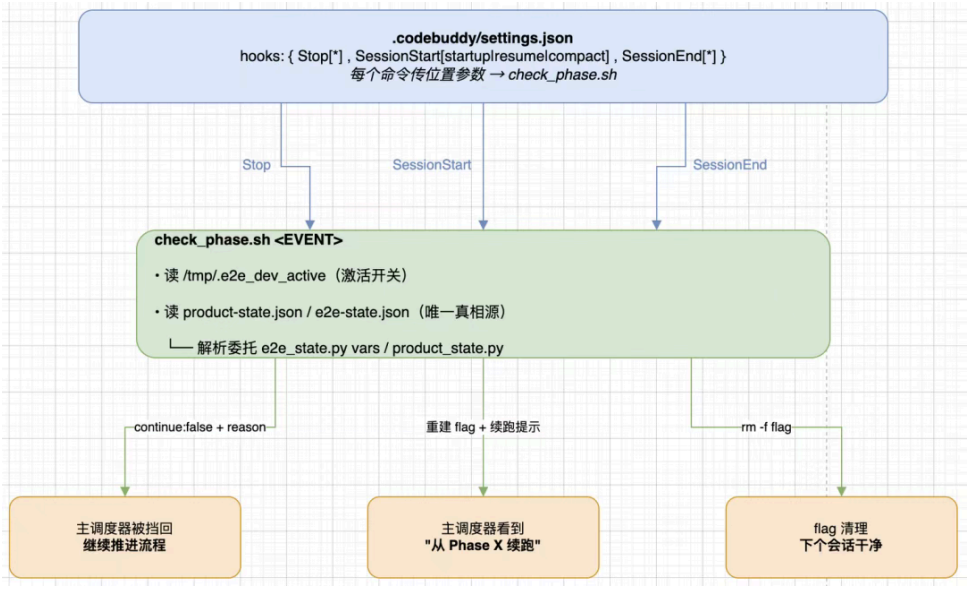
每个子 Agent 在独立上下文中执行完毕后，将其执行结果写入状态文件的对应字段。主调度器不依赖对话记忆，而是直接读取状态文件来确定"现在到哪一步了、下一步该干什么"。状态文件落盘，实现流程可中断、可恢复、可观测。



4.1.2 激活标志 + hook 机制：强制流程推进

光有状态文件还不够——AI 可能在流程还没结束时"偷懒"停止，为了进一步确保流程能够按照状态文件完整驱动执行，并且每个子 Agent 在执行后能够准确地将状态和执行结果落到状态文件中，我们还引入了 hook 机制，通过在三个事件（`Stop`、`SessionStart`、`SessionEnd`）中注入脚本逻辑确保流程的确定性：

- `Stop Hook`：流程不可被主调度器"偷懒式"提前停止
- `SessionStart Hook`：跨会话断点可自动恢复
- `SessionEnd Hook`：会话结束后清理残留状态，不污染下一个无关会话



4.2 专家 Agent 体系：每个专家只做一件事

将流程拆分成多个步骤后，最直接的做法是让同一个 Agent 依次执行每个步骤。但实践中发现有两个问题：

- Agent 在角色频繁切换时行为不稳定——它可能在开发步骤里越权去修改协议，也可能在审查步骤里顺手改了代码。
- 不同步骤对模型能力的要求差异很大：简单的状态解析用便宜模型即可，复杂的需求拆解则需要更强的推理模型。

因此，我们在 Agent 层基于以下原则设计了一套专家 Agent 体系：

1. **单一职责**：一个 Agent 只干一件事。code-reviewer 只评不改、interface-verifier 只诊断不改、code-fixer 只在收到问题清单后才动手。
2. **上下文隔离**：每个 Agent 在独立上下文执行，只看到自己该看的输入。既避免注意力稀释，也避免上游噪音污染下游判断。
3. **工具最小权限**：按职责裁剪工具集，审查类不给写文件权限，规划类不给发布权限，从机制上禁止越权操作。
4. **确定性的输入输出**：Agent 之间不靠"对话"传递信息，而是靠**结构化状态文件**。输入是明确的字段，输出也是明确的字段，可被脚本解析、可被断点恢复。 5.**模型可插拔**：职责边界清晰后，每个 Agent 可独立选模型：简单步骤用更便宜的模型，复杂推理用更强的模型

通过**角色定义 + 工具约束 + 知识隔离**，我们为每个阶段设计专门的专家 Agent：

类别	Agent	职责
规划智能体	`product-analyst`	产品需求拆解 → N 个子需求 + `kb-query` 分析 + 多轮澄清 + 建子 story + 初始化 `product-state`
	`requirement-analyst`	需求澄清 + 技术方案 + 初始化 state
	`task-planner`	任务拆解 + DAG + 并发/串行编排 + 全局变更确认

类别	Agent	职责
执行智能体	`proto-engineer`	Rick 平台 proto 变更 + 桩代码同步
	`backend-developer`	单任务 worktree 开发、波收口 merge、 集成收口接线编译
	`code-fixer`	复用型修复：P4 改完只编译+回归单测；P7 改完编译+发布
验证智能体	`unit-tester`	codegen 单测 + 增量覆盖率
	`interface-verifier`	`trpc-gateway` 调接口 + 断言 + galileo 根因分析（只诊断不改）
	`test-case-designer`	设计用例，识别字段三分类
审查智能体	`code-reviewer`	codar-local 评审，输出 BLOCKER 清单（只评不改）
集成智能体	`publisher`	dtools 发布 test + 框架配置重启
	`git-committer`	用户确认后提交工蜂 + push + 发起 MR

其相比较于直接 call 起子 Agent 执行的方式，在实践中带来了三个收益：

- 通过角色定义，使 AI 执行的行为更加稳定。
- 通过预先内置好 Prompt，避免每次执行都需要主调度器临时给 Prompt 片段，减少调度器的负担和抖动。
- 便于独立维护和更新迭代：可以单独修改某个专家的行为而不影响其他流程；针对不同专家，可以选择不同的模型来执行。

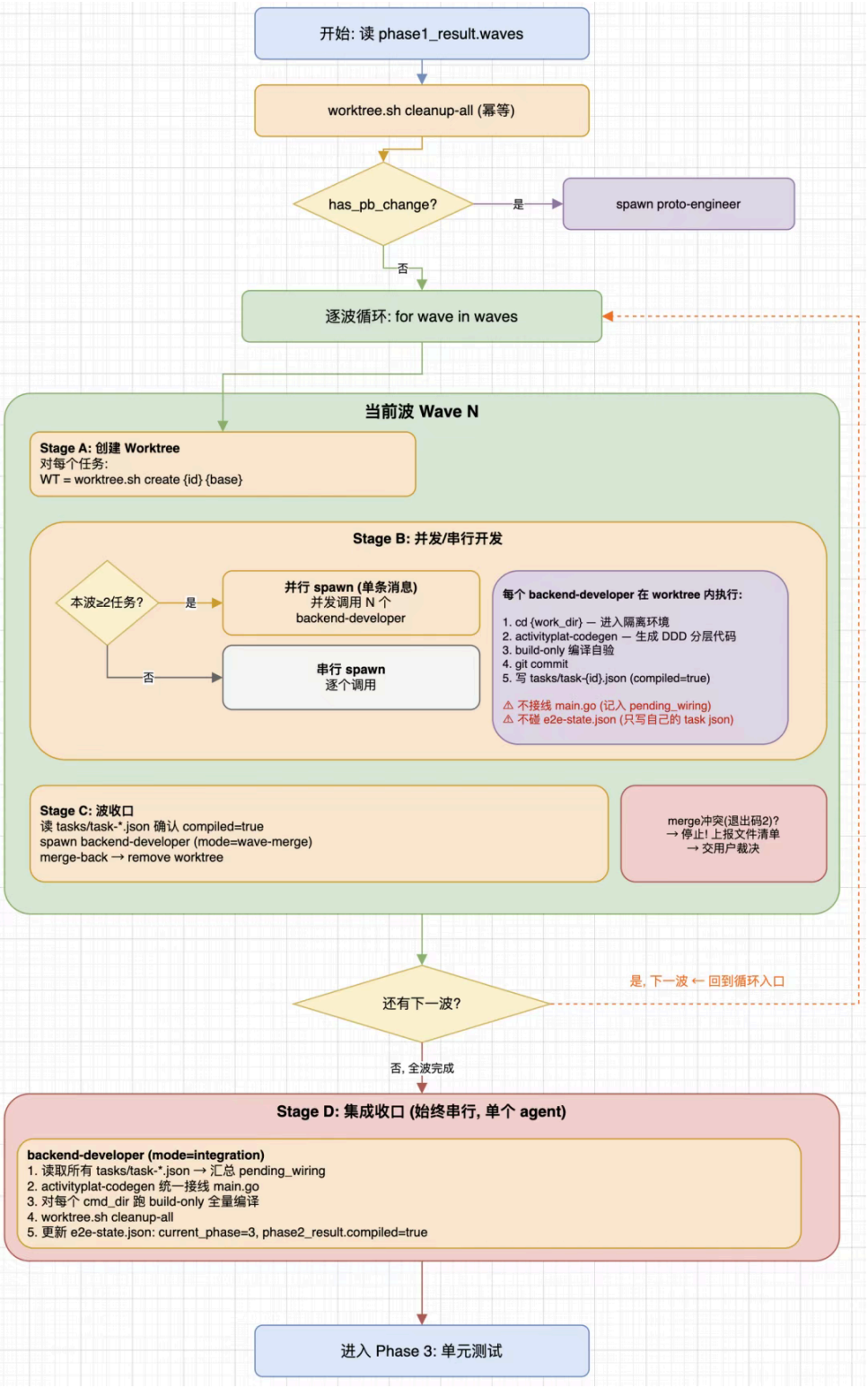
4.3 从串行到并行：DAG 编排 + Fork-Join

当一个业务需求被拆分成了若干子需求，每个子需求都被拆分成多个开发任务后，如果全部串行执行，整个执行流程耗时会非常的长。同时，主流程下积累的上下文信息也会快速膨胀。为此，我们从两个维度做了优化：

4.3.1 Worktree 隔离：同一需求内的多任务并行

即使单个需求内部，多个独立接口或模块之间也往往没有先后依赖，完全可以同时开发。

我们定义了一个 `task-planner` 的 Agent，专门负责任务拆解与并发编排规划，其会根据需求文档和实现方案，按照接口、功能模块进行任务拆解，并针对每一个任务标注其与其他任务的依赖关系，以及预测需要修改的目录文件，并将所有任务列表按照依赖关系构建成 DAG 拓扑分层，同一层的内的任务可以并发执行。 主调度器会根据 `task-planner` 返回的拆解结果进行编排，并发执行的任务分配不同的 `worktree` 给到负责开发的 Agent，并在本轮所有开发 Agent 执行完成后，统一执行 `merge` 操作收口。



4.3.2 Fork-Join：多需求的并行开发与统一收口

当一个产品需求涉及的逻辑非常复杂时，我们往往都会先将需求拆分成多个 story 单，再将每个 story 单依次完成。如果需求涉及多个人一起开发，由于整个需求需要一起提测，我们往往会拉一个临时分支，大家把开发的代码都先往上合，然后统一将这个分支发布到测试环境进行测试。

这里我们的实现也类似人协作的方式，在原来流程的基础上，引入了一个最前置的需求拆解阶段，这一阶段会结合建设的业务知识库，对需求进行分析，并经过与人的多轮交互澄清，将一个完整的产品需求拆分成多个子需求。

之后，调度器会进入 Fork 并行开发阶段，执行从任务拆解到代码 CR 的阶段，其中拆解的多个任务，仍然按照上面的流程并行开发：

```
Phase 1: 任务拆解 → Phase 2: 波次开发(并发) → Phase 3: 单元测试 → Phase 4: 代码审查
```

当所有代码开发完成后，调度器进入 Join 阶段，完成后续的流程：

```
Phase R: 产品需求拆解
  ↓
Fork 段（并行）：每个子需求各自跑 Phase 1~4
  ↓
Join 段（串行收口）：合并 → 发布 → 验证 → 提交
```

4.3.3 冲突治理：事前隔离与串行收口

并行不是把任务拆开就完事，多个 Agent 同时修改代码，不可避免会产生冲突。如果冲突处理不当，会导致编译失败、代码被错误覆盖等问题，冲突如何解决也是工程上的一个难题。我们对四类冲突分别制定了解决策略，其核心思路就一个：**能事前隔离的就事前隔离，必须共享的就串行收口。**

冲突类型	策略
Merge Conflict	由 task-planner 的 touches 做事前的文件级隔离，尽量让同批次任务不碰同一文件；真发生冲突时正常解决冲突，绝不用 --no-verify/丢弃方式绕过
Shared file (如 main.go 等)	不在并行执行中修改，统一收敛到集成收口阶段由单个 Agent 执行，避免多 Agent 修改同一入口
Proto 协议修改	协议变更前置且串行：仅当 has_pb_change=true 时由 proto-engineer 在 Rick 平台统一变更、统一生成桩代码，下游开发 Agent 基于已生成的桩代码开发
DB/配置变更	全局变更**一次性前置确认**（task-planner 阶段汇总），落地由专门 Agent 执行，避免并行更改数据库、修改配置

4.4 脚本化执行：确定性操作交给脚本

在跑通流程之后，我们复盘发现一个现象：整个端到端流程中，大量步骤其实是确定性操作——比如解析状态文件的 JSON、创建 git worktree 、执行编译命令、调用发布工具等。这些操作不需要推理，只需要执行。让 AI 来做这些事，不仅白白消耗了 token，还引入了不必要的随机性——AI 可能写错 shell 语法、用错参数、或者在无关步骤上发散，同时，同一需求多次跑、不同模型跑，token 消耗和结果也都可能不一致。

因此我们得出一个重要结论：**AI 负责认知，脚本负责执行**。AI 的价值在于判断、分析、生成这类需要想的动作，而确定性的执行操作，应当交给脚本。

我们将整个端到端开发流程固定的一些操作步骤流程脚本化，前后沉淀了接近 15 个脚本，将每个流程中，一些具有明确输入输出、或者操作路径非常清晰的步骤，都改为用脚本的方式来

执行，在消除执行流程不确定性的同时，也能在这些环节省下不必要的 token 消耗：

- 状态机解析执行脚本 `e2e-dev.py`：把主调度器原本靠 AI 实现的状态文件读取解析和下一步状态确认，改为通过调用脚本直接拿到对应的结果。
- 多任务并行开发 worktree 脚本 `worktree.sh/sub_worktree.sh`：在多 Agent 并发开发同一个需求的多个任务时，将 `git worktree` 相关的创建/ merge /多分支集成/清理等操作，都封装成脚本直接一键执行，避免 AI 来回切换工作空间了解背景信息后再操作的成本。
- 编译发布脚本 `build-and-publish.sh`：将服务编译发布抽取成一套通用的脚本，将原本由 AI 按照目录探索发布脚本、或自己根据 dtool 工具说明实现编译发布的方式，改为调用通用脚本一键发布。
- 知识库一键初始化/更新脚本 `kb-init.sh`：每次（或首次）使用知识库进行检索前，需要在本地进行初始化或更新操作。
- ...

4.5 打通最后一公里：将 AI 嵌入 DevOps 全流程

端到端开发的流程远不止写代码——一个需求的完整交付，还涉及 TAPD 创建子需求、Rick 平台修改协议、123 平台发布服务、七彩石修改配置、伽利略查看日志等等。如果这些环节都需要人手动在不同平台之间切换操作，AI 在流程中就只能起到局部加速的作用，无法实现真正的端到端自动化。

因此，我们在工程中将公司内各平台的能力进行了系统化集成，让 AI 在流程中经人工确认后，可以直接完成跨平台的读写操作。

4.5.1 tRPC-Gateway：让本地 AI 能调通内网接口

后台服务部署在 123 平台上，属于内网 idc 环境，但是 AI 的执行是在本地电脑上。如何实现 idc 环境调用这个问题也困扰了我们，针对每个服务都申请白名单的方式显然不够通用。

4.5.2 Codar：AI 写的代码，让 AI 来审

AI 自身缺乏全局视角，对结合业务逻辑边界的隐含 bug 识别能力弱。对此，一种比较直接的方式是：自己沉淀一套与业务逻辑相关的 CR 规则集，并随着业务的迭代持续更新其中的规则信息。但在活动平台系统重构之初，我们团队就和 Codar 团队进行了合作，在代码 MR 环节引入了 Codar 团队的 CR 流水线，实践下来我们发现 Codar 对代码隐含的业务逻辑漏洞的识别能力非常不错，因此，我们和 Codar 团队进一步合作，集成了 Codar CR skill 到我们的 CR 步骤中。

CR 环节，AI 除了会根据现有的代码规范对代码进行 CR 外，还会通过 Codar 提供的 CLI 工具对增量代码进行 CR，并将 CR 出来的问题和风险级别返回给主调度器，由 Fix Agent 针对问题进行修改。

4.5.3 多平台读写集成

在一个需求的开发流程中，涉及 TAPD 平台创建子需求单、Rick 平台更新协议、123 发布服务、七彩石修改配置、伽利略监控等等。目前来说，大部分平台都提供了相应的 MCP 服务或

者 Skill，来实现 AI 集成，基本也可以在公司**Knot** 平台 还有**03** 平台上找到这些功能。

目前我们的端到端开发工程也都基本集成了上述平台的能力，AI 在整个开发流程中经人工确认后，可以直接在相应的平台上执行读取/写入操作。

> 不过这里有一个例外：123 平台上现在默认的业务配置，嵌入的配置页面走的是七彩石的 tconf 集群，但七彩石的 mcp 并没有支持 tconf 集群。因此很多服务如果是直接在 123 平台业务配置这里新增配置，是没有办法走七彩石的 mcp 来实现服务配置的读取和写入的。

五、复盘、规划与思考

5.1 实践复盘：核心工程原则

在我们团队的工程实践中，经过多轮调试与架构演进，总结出以下核心经验与优化建议：

应对异构服务拓扑，构建目录解析抽象层：实际业务中的代码组织形式复杂多样，涵盖单仓单服务（Multi-repo）、单仓多独立服务目录，以及共享 `go.mod` 与底层公共包但具备多 `main.go` 入口的 Monorepo 架构。直接依赖 Agent Skill 来兼容这些异构拓扑会成倍放大调试成本。建议在工程架构初期，优先建立统一的目录解析抽象层，以标准化后续的处理流程。

调度架构演进：从 Agent 驱动转向强类型代码编排 ： 早期我们团队在基于 Claude 与 Skill 机制构建更新流时，就已经发现存在指令依从性不足的隐患。为控制成本引入 hy3-preview 后，进一步暴露出流程失控（如子 Agent 越界接管主链路）及调试周期过长等工程痛点。为此，我们在知识库工程上优先做了两个调整：

- **弃用主子 Agent 模式**：改由外部主程序编排全局流程。通过代码直接调度 Codebuddy CLI，实现职责解耦：hy3-preview 专注于局部服务文档的生成，Claude 则负责高维度的域（Domain）及全局架构综述。
- **弃用 Shell 脚本**： 测试的时候发现，大模型生成的 Shell 脚本常潜藏隐性语法错误或边界逻辑漏洞，且往往在长链路末端才抛出异常，极耗排查时间。最终将驱动层全面重构为 Go 代码，利用强类型语言的特性来保障执行控制流的严密性与稳定性。

严格禁用项目级 Memory 以保障状态隔离：全局 Memory（记忆）机制会引入不可控的上下文串扰，导致 Agent 偏离既定职责（如越界干扰主流程），并严重破坏单一子任务的幂等性与可复现性。在构建强确定性的自动化流程时，必须彻底关闭该功能，以维持每次任务上下文的纯粹性。

引入 Mock 机制，分离调度逻辑与生成耗时：自动化构建流水线主要由“流程调度”与“文档生成”构成。其中，调度逻辑最为复杂，而生成任务耗时最长。及早在工程中引入 Mock 机制，通过模拟或跳过大模型的实际生成环节，能够以极低的成本快速验证、迭代调度链路，大幅提升整体联调效率。

如果说上面这些是具体场景下的经验，那么把它们再往上抽象一层，我们沉淀出了几条贯穿整套 Harness工程的核心原则——它们既是前面所有设计的出发点，也是我们判断"一个环节该

不该这么做"的准绳：

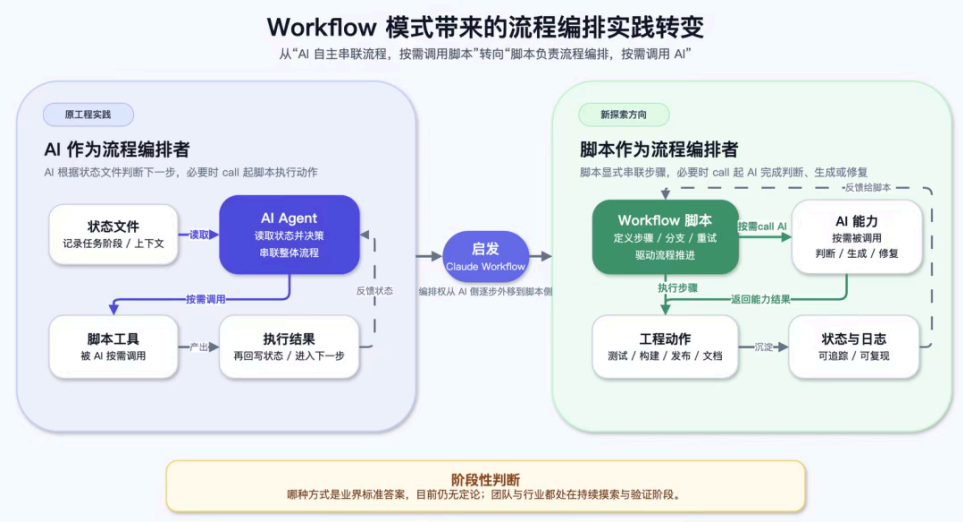
- **AI 负责认知，脚本负责执行**——工程的核心思想
- **长链路必须状态化**——状态脱离上下文落盘，才能稳定续传
- **知识库必须结构化**——业务知识结构化，AI 才能精准检索
- **Agent 必须职责隔离**——单一职责、上下文隔离、最小权限
- **执行步骤必须脚本化**——执行问题不要交给推理
- **Workflow 比 Prompt 更重要**——流程编排的确定性，胜过反复打磨 prompt
- **终局认知**：未来比拼的不是"用了多少 AI"，而是能否把 AI 当作一个工程系统来设计

5.2 下一步：从"能跑"到"跑得好"

目前来说，对于这套 Harness 工程，仅仅处于"能跑"的刚起步状况，在很多地方、细节方面还有待完善和改进：

- 缺少自我复盘迭代的自进化能力，每次跑完一个流程后，依赖人工发现并修复流程中的问题。
- 缺乏一套完善的评估体系，来评估整个AI 端到端开发流程的稳定性、开发效果、token 成本消耗等等。
- 整套体系重度依赖于 codebuddy cli 工具来运行，还没有实现工程与工具的解耦。
- ...

最近，Claude 发布的 Workflow 模式，也给我们带来了新的启发，团队也正在尝试将流程编排引入到这套工程实践中，将这套工程由原来 AI 自己根据状态文件实现流程串联，必要时由 AI call 起脚本调用的流程，改为由脚本实现流程串联，必要时由脚本 call 起 AI 调用的方式来实现。至于说到底哪种方式才是业界的标准答案，目前也没有定论，大家都在摸索，但方向是一致的——**让确定性的归脚本，让认知的归 AI**：



5.3 一些开放性思考

5.3.1 TDD 在 AI 时代：先写测试还是先写代码？

TDD 是软件工程中令人向往的开发模式，但在实际代码开发中践行很难——接口字段、校验规则、依赖接口，甚至很多业务逻辑，往往都要等代码和协议落地后才能确定，提前写单测极易返工。

因此，在我们的实践中，TDD 的落地不在代码层面，而是集中在接口测试用例上：在一开始的需求方案设计环节，便会根据需求，生成一份**需求级测试用例**，包含输入输出的预期。在流程后面的请求验证环节，便会基于这份测试用例，构造对应的请求，并针对结果判断是否符合用例预期。

5.3.2 AI 工程的架构分层：我们缺的不只是好 Agent，还有好架构

传统软件工程中，代码如何组织，架构如何设计，业界有着非常多非常成熟的方法论，MVC、DDD、Clean Architecture 等等。但是对于 AI 工程，架构上如何设计，Agent、Skill 之间如何组织， AI 工程与 AI 工具如何解耦，Agent 与 Skill 如何插拔式组合等等，当下似乎并没有看到什么很成熟的方法论或实践规范，更多的都是结合 Code 工具的规范，定义一个 skill 目录，一个 agent 目录，然后就都往里加。当下我们这套工程中的组织方式，也有着同样的问题。工程与工具的解耦、Agent/Skill 的分层与插拔式组合，是后续要重点优化的方向。

5.3.3 代码还重要吗？——从"代码为王"到"架构为王"

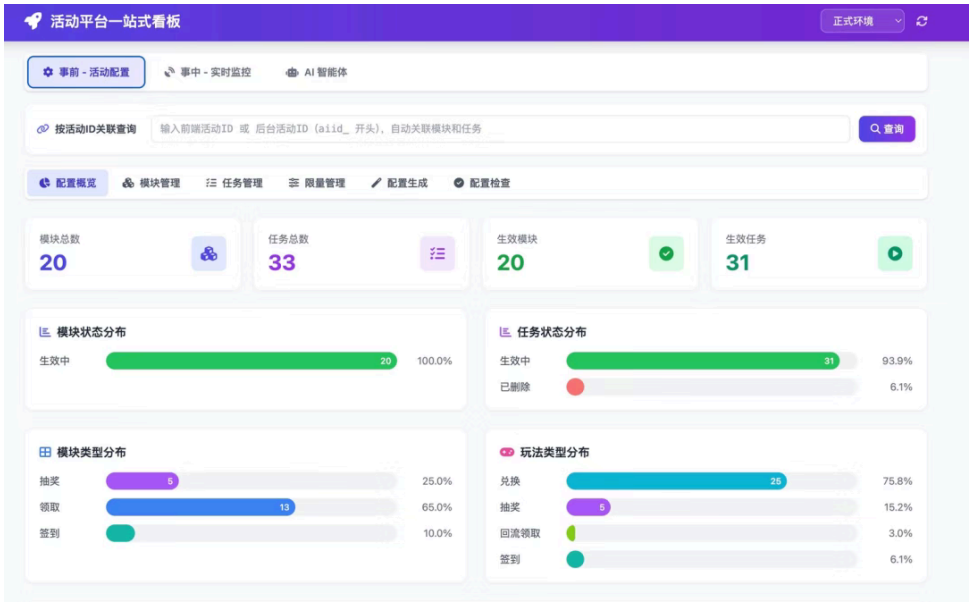
在 AI 生产代码效率越来越高的当下，有一种说法是“忘记代码的存在”：人不再逐行审查 AI 写的代码，只关心运行结果是否符合预期。YouTube 上有一个来自 Anthropic 研究员的演讲，分享的就是这种观点：**Vibe coding in prod**

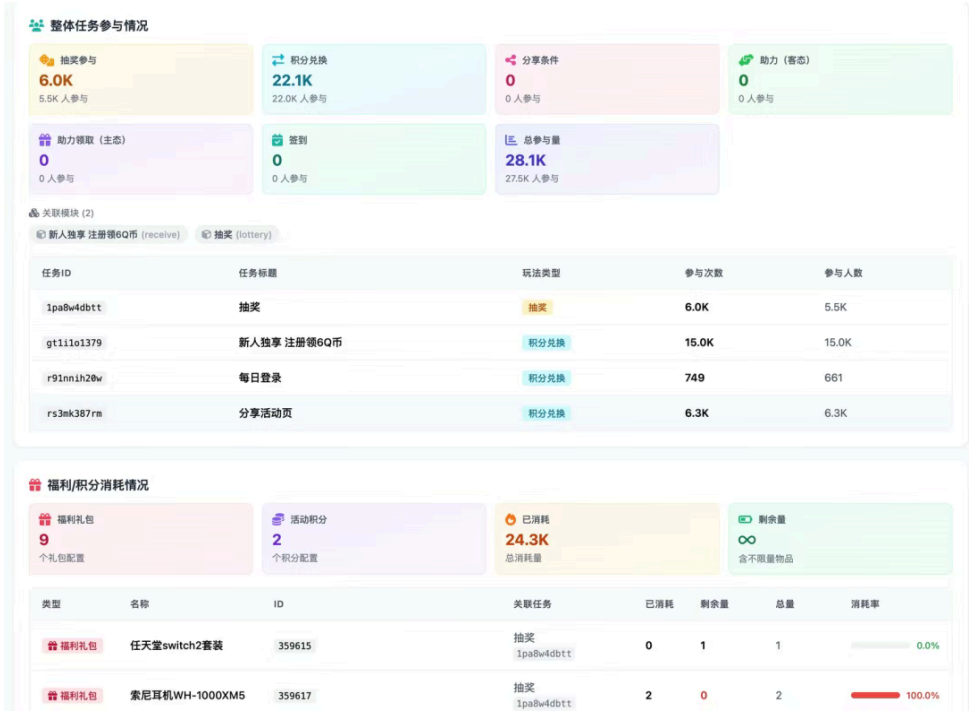
但是，我们所面对的并不是一个工具系统，而是一个涉及多微服务、调用链路复杂、有着高并发、高性能要求的复杂在线业务系统，就现阶段来说，我认为保持高质量的代码架构依旧非常重要：

1. ai 代码腐化速度非常快，哪怕是现在 Claude Opus 4.8，放任迭代会很快偏离既定的规范。
2. 高质量的代码和架构会反哺 AI，让 AI 生成的代码质量更高，这是一个正反馈的过程。
3. 一旦人对代码失去掌控，线上出问题就只能完全靠 AI 排查定位。

但不能否认这个观点的前瞻性，在我们团队内部也有类似的实践：重构后的活动平台系统，在 With 平台搭建了一套统一的集成看板，集成了包括在线活动一览/活动实时参与情况监控/活动消耗实时监控/活动诊断/活动复盘等能力，实现从事前到事后的全活动生命周期一站式运营。

这套看板代码 100% 由 AI 通过对话式 Vibe Coding 生成，没有一个人去 CR 过它的代码，前端/后台/产品/运营，任何一个同学，只要有任何活动看板的需求，都可以在这上面通过对话的方式集成。





这其中的区别在于：**看板这类"结果导向、容错率高、无强一致性要求"的系统，适合黑盒化；而核心在线业务系统，现阶段仍需要人守住架构这条线。**

而这条线，或许也会随着 AI 的进化不断后撤，或许终有一天，代码会像汇编语言那样，从开发者手中精心雕琢的核心资产，退场为智能体之间无声流动的中间表达——被生成，被消费，却不再被凝视。

从"逐行审查"到"忘记代码"，变化的从来不是代码的价值，而是人在系统中的位置。这条边界还会继续移动，而我们能做的，是始终想清楚：此刻，人究竟该站在哪一侧。

文末附：团队产品推荐——Mac 应用宝

🌟 Mac 应用宝核心亮点

✅ 端游大作，Mac也能玩3A

支持Steam及端游exe自上传运行，Mac终于不再与3A大作无缘。端内搜索「Steam」即可获取完整Windows版下载上传运行指引，优先支持Steam及Steam单机游戏，其他端游也可尝试。

✅ 原生体验，点击即玩

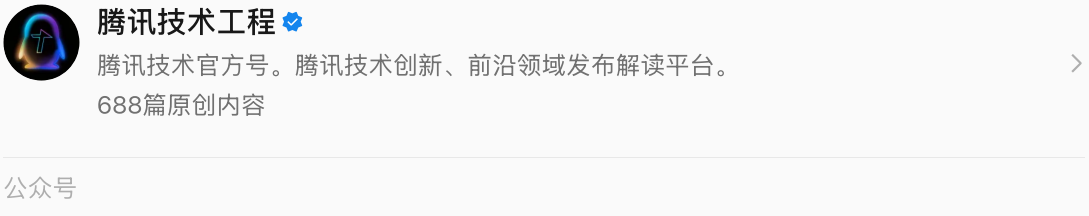
APP图标一键启动，无需繁琐配置。交互设计极致还原Mac原生操作感，打开即玩，流畅自然。

✅ 极致性能，颗秒爽感

支持4K + 120帧 + HiDPI视网膜屏，充分利用Mac屏幕潜力。键鼠深度定制适配，每一帧都是享受，每一次操作都精准跟手。

📥 下载：**https://sj.qq.com/**







阅读 2.1万

留言

写留言

还没有留言，来写首评